

Application Security Assessment & Remediation Report

Target: Shelfie RESTful API

Lead Analysts: Magda Dominguez, Ogechi I., Ri K., Monika P., Ruth T.

TABLE OF CONTENTS

1. Understanding Common Threats	4
1.1 Common Web Application Vulnerabilities	4
1.2 Vulnerabilities Present in Shelfie API	4
2. Secure Coding Practices	8
2.1 Coding Guidelines	8
2.2 Current Security Controls	8
2.3 Improvement Roadmap	8
3. HTTPS Implementation	10
3.1 Importance of Implementing HTTPS	10
3.2 Role of SSL Certificates in Ensuring Secure Communication	10
3.3 Obtaining and Installing an SSL Certificate	11
3.4 Demonstrating Enforcing HTTPS in a Flask Application	12
4. Authentication and Authorisation	13
4.1 Theoretical Frameworks	13
4.2 User Authentication	13
4.3 Authorisation and Access Control	14
4.4 Flask: Login Management	14
5. Protecting Your Application Data	15
5.1 Theoretical Foundations of Database Security	15

5.2 Best Practices for Securing User Data	15
6. Logging and Monitoring	16
6.1 The Role of Logging and Monitoring	16
6.2 The Purpose of Logging in Recording Critical Events	16
6.3 How Monitoring Tools Contribute to Threat Detection	16
6.4 Additional Tools	18
6.5 Handling Software Updates and Patches	18
References (IEEE)	19

TOPICS

1. Understanding Common Threats

1.1 Common Web Application Vulnerabilities

The OWASP Top 10 [1] lists the most common vulnerabilities in web applications, and the ones below are the ones found in Shelfie API [77], with CWE IDs [2] where relevant.

SQL Injection (CWE-89) is when an attacker sneaks malicious code into an input field so the database runs commands it shouldn't, exposing, changing or deleting data [52].

Cross Site Scripting (CWE-79) is when an attacker injects a script into a page that then runs in

another user's browser, which can lead to data theft or session hijacking [53].

Broken Access Control (CWE-639), usually an insecure direct object reference, is when the app doesn't check a requester is actually allowed to access a resource, so a user can view or edit someone else's data just by changing an ID [3].

Broken Authentication (CWE-287) covers weak password rules, no protection against repeated login attempts, or badly managed sessions, any of which let an attacker get in without the real password [1].

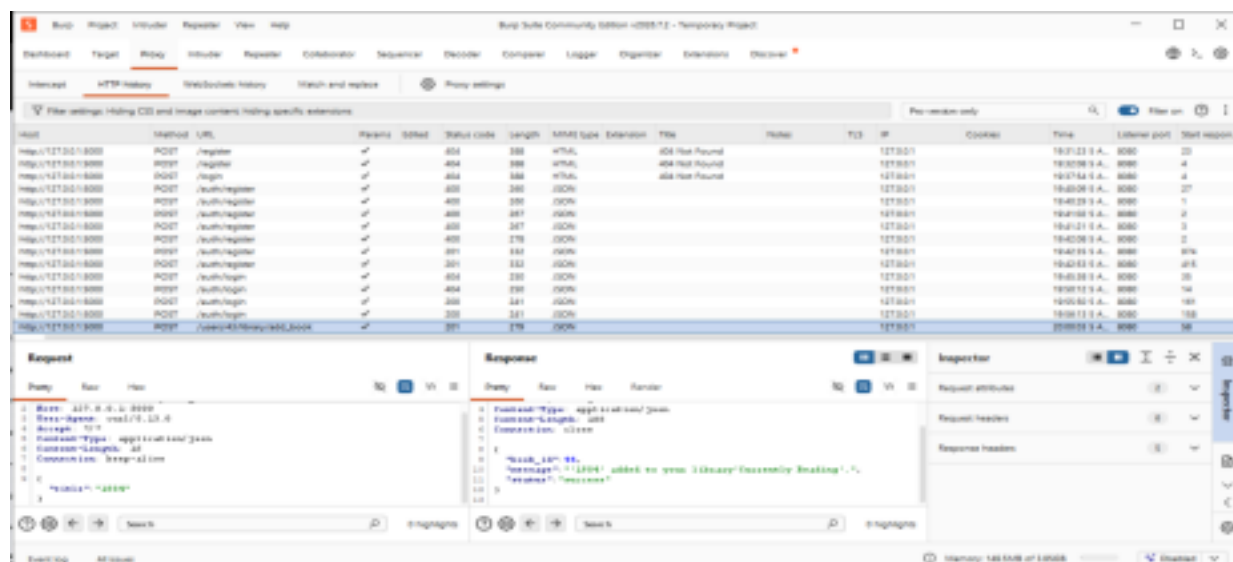
Security Misconfiguration means things like debug mode left on in production, or errors that expose internal details to users. Flask's own docs warn its debugger can let arbitrary code run from the browser if it's left on [4].

1.2 Vulnerabilities Present in Shelfie API

Testing app.py, utils.py and db_utils.py with Burp Suite confirmed several of these.

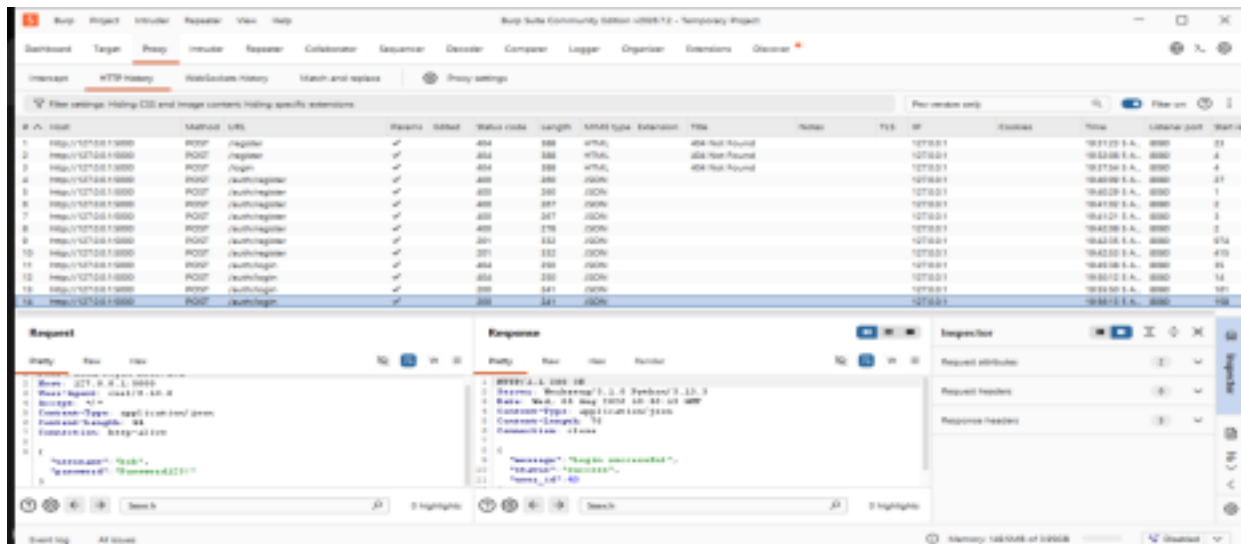
Broken Access Control (CWE-639) is the most serious issue. Routes handling a user's library, profile or reviews take user_id straight from the URL, e.g. [PUT /users/<user_id>/library/<book_id>](#), with no check that the requester actually owns that ID. This was confirmed by registering two accounts, Alice (42) and Bob (43), then sending a POST to /users/43/library/add_book with no session token attached. It succeeded, adding a book to Bob's library and proving the API trusts whatever user_id is in the URL.

4



Burp Suite: IDOR – writing to another user's library (POST /users/43/library/add_book, no auth)

Broken Authentication (CWE-287) is partly mitigated: passwords are hashed with Werkzeug, so there's no cryptographic failure in storage. But login has no rate limiting, and a successful login just returns the user_id with no session token, e.g. Bob's login response was simply **"success"** and user_id 43, nothing in the headers. That makes user_id the only thing tying a request to an identity, and as the access control test showed, it's trivial to forge.



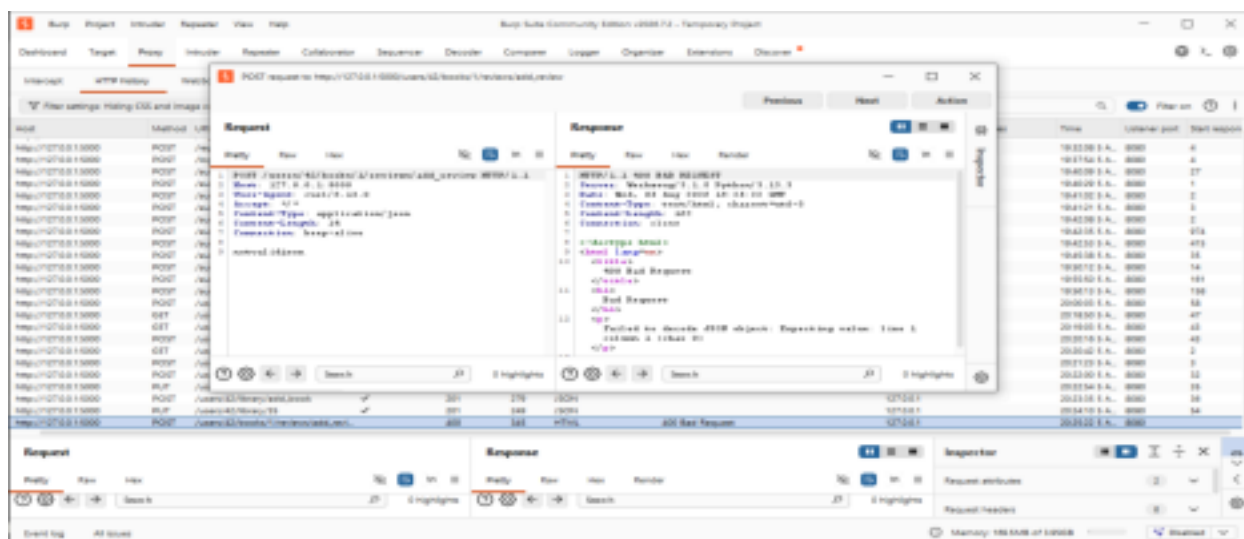
Burp Suite: Broken Authentication – login response with no session token (POST /auth/login, user_id only)

5

Security Misconfiguration shows up twice: app.py runs with debug=True, which exposes Flask's interactive debugger, and several routes return raw exception strings that leak internal detail. Running app.py showed an active Debugger PIN in the terminal, and sending malformed JSON to **POST /users/42/books/1/reviews/add_review** returned a raw Werkzeug error instead of a generic message.

```
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 120-454-171
```

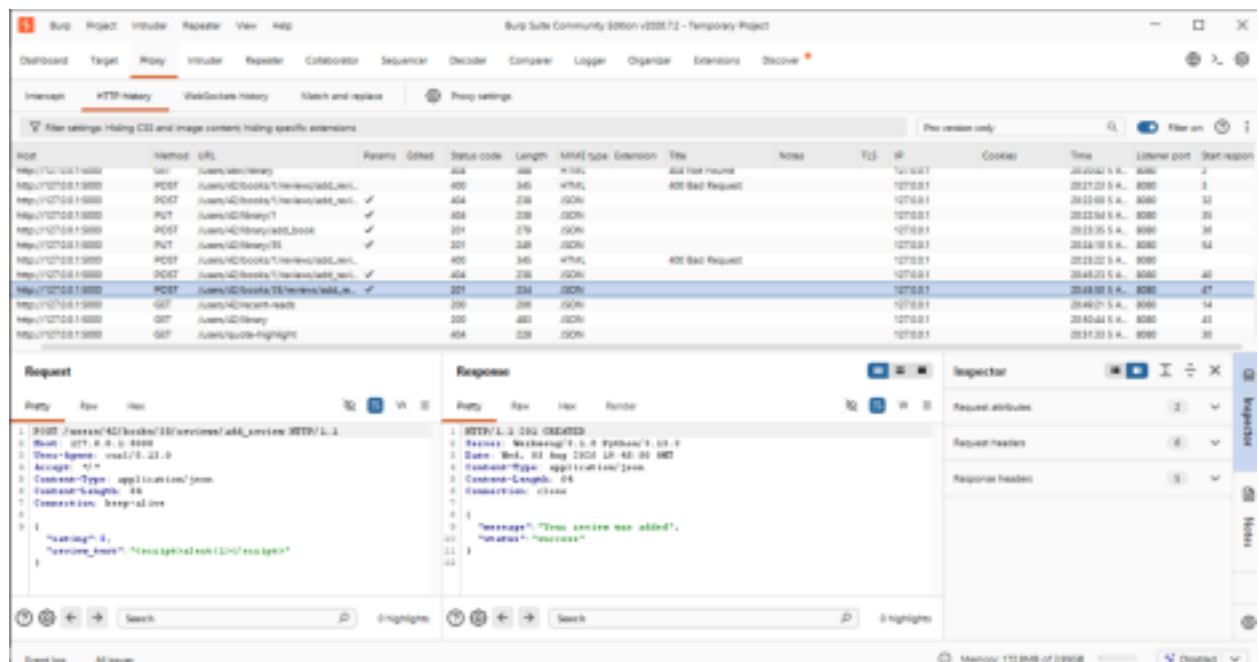
Active Debugger PIN



Burp Suite: Security Misconfiguration – raw exception leaked (POST /users/42/books/1/reviews/add_review, malformed JSON)

SQL Injection (CWE-89) is largely mitigated. Every query uses parameterised placeholders, apart from LIMIT values, which are inserted via f-string but wrapped in int(), so there's no reachable attack surface to test here even though the pattern itself isn't ideal.

Cross Site Scripting (CWE-79) is a potential risk rather than a confirmed one. The API accepts and stores a script payload without sanitising it, but since the backend never renders the stored content, it can't actually be triggered yet. It would become exploitable the moment a frontend displays this data without proper output encoding.



Burp Suite: Potential XSS – malicious script payload accepted by API

For Shelfie specifically, the IDOR issue means one user could read or alter another user's private quotes and reviews, which is exactly the personal content the app is meant to protect. Since there's no session-based authentication tying requests to identities, this risk applies to every user by default.

2. Secure Coding Practices

2.1 Coding Guidelines

As a backend-only REST API [5], Shelfie enforces strict endpoint isolation and data validation across all current and future controls.

2.2 Current Security Controls

A four-layer Defence in Depth architecture decouples presentation routes from database access [1]. To prevent SQL injection, parameterised queries with tuple placeholders (%s) in db_utils.py force MySQL to compile structures first, treating inputs as literal data [6]. Context Managers in db_connection.py safely close sockets, preventing DoS leaks, while MySQL binds to localhost to secure port 3306 [28].

Plaintext passwords are hashed using Werkzeug in utils.py prior to storage [7]. Input validation uses Regex for email formats and password complexity, alongside whitespace sanitisation [12].

OOP alternative constructors in models.py enforce strict type casting and structural integrity upon entry, while 500 errors intercepted in app.py return JSON to block stack trace exposure [27].

2.3 Improvement Roadmap

Architecture & Data:

- **Security Misconfiguration & Data Leaks:** Disable debug mode [4] and move secrets to environment variables [51].
- **SQL Injection (CWE-89)** : Migrate to SQLAlchemy ORM to minimise human error [6].
- **Remote Code Execution:** Validate payloads with Pydantic [11],[64] and implement automated dependency scans [65] to patch vulnerable packages [59],[62] and prevent exploitation [60],[61].

Identity & Access:

- **Zero Trust & RBAC:** Enforce Role-Based Access Control [67] to strictly manage permissions.
- **Credential Protection:** Upgrade to Argon2id [7] against GPU attacks [66] and remove password length caps per NIST [9].

- **Data in Transit:** Enforce TLS 1.3 encryption [68].

- **IDOR (CWE-639)**: Shift to contextual routing (e.g., [/user/library](#)) via decoded stateless JWTs [63],[70],[71], standardising header-based authentication [73].
- **Session Hijacking**: Future web expansions must encapsulate JWTs within HttpOnly, Secure, and SameSite=Strict cookies to mitigate XSS [54].

Mitigation & Monitoring:

- **Credential Stuffing (CWE-287)** : Flask-Limiter [75],[76] to throttle login attempts [1]. ●
- XSS (CWE-79) & Bypasses**: Strict JSON [8],[72] and model-pipeline Unicode normalisation (NFKC) [74] to block homograph attacks [10].
- **Visibility**: Log access patterns via Unicorn/Nginx [69] and monitor anomalies with Grafana [17].

3. HTTPS Implementation

3.1 Importance of Implementing HTTPS

Hypertext (or HyperText) Transfer Protocol (HTTP) are resource requests made by a client, (e.g. browser or API consumer) to a server - the Shelfie API. The requested data is transmitted over HTTP and separated into 'packets'. These packets are susceptible to being 'sniffed' - viewed or accessed - by attackers.

Hypertext Transfer Protocol Secure (HTTPS) is a secure, extended version of HTTP [20]. HTTPS ensures the confidentiality and integrity of the data exchanged between client and server and authenticates the server's identity to the client. Furthermore, it ensures regulatory compliance and ensures user privacy through encrypting transmitted sensitive data [40].

Not having up-to-date encryption via HTTPS, leaves the Shelfie API vulnerable [23]. The key threat is that attackers can intercept and steal data, with an increased likelihood when communication is transmitted over insecure mediums such as public WiFi [23], [24].

The recommended change for the Shelfie API is to encrypt through HTTPS; HTTPS uses hashing and terminates the client-server connection if modifications are detected. This reduces the risk of an attacker harvesting intercepted Personally Identifiable Information (PII), which could later be used to craft a targeted social engineering attack such as whaling[18]. An attacker could intercept unencrypted API traffic over the network to obtain user's PII and favoured quotes from the Shelfie API, and potentially deduce passwords.

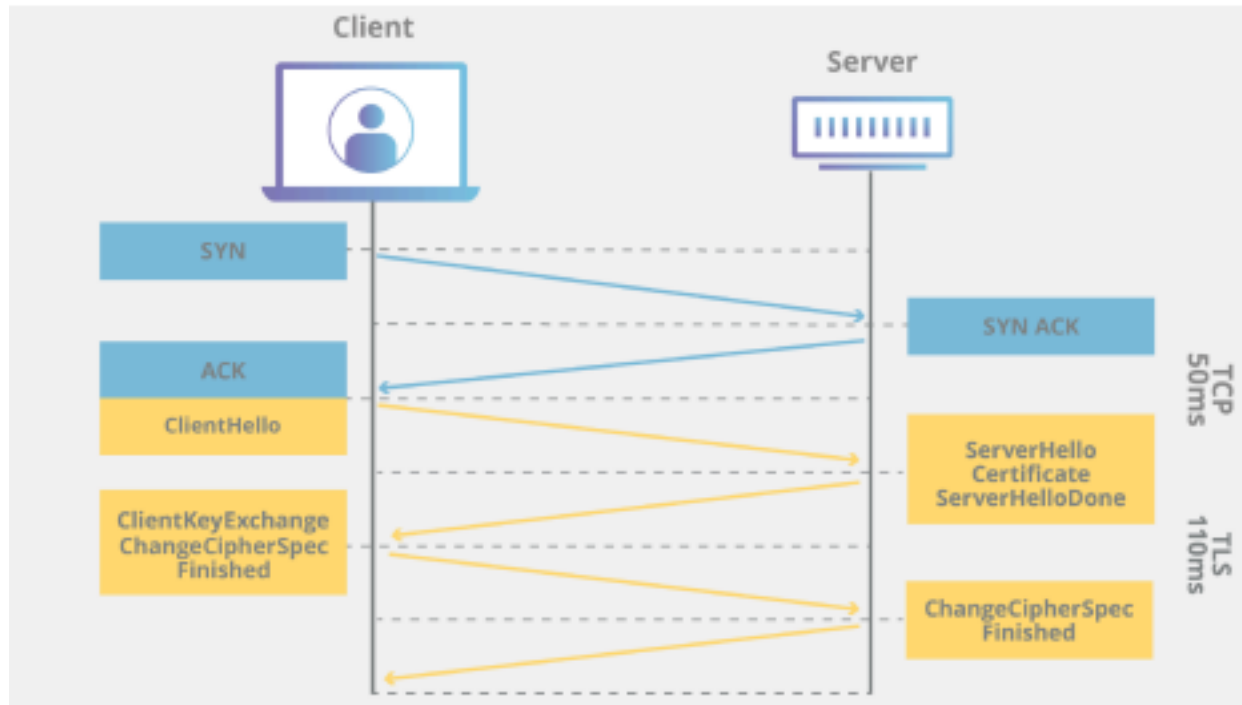
For future integrations with a web app, HTTPS also furthers performance and analytics, as HTTPS web applications load faster and can track referral links better than HTTP [20].

3.2 - Role of SSL Certificates in Ensuring Secure Communication

Secure Socket Layer (SSL) certificates authenticate the server's identity and enable an encrypted connection [25]. Transport Layer Security (TLS) is the updated and secure version of SSL, with the term SSL remaining in use [32]. SSL is a security protocol describing the use of algorithms and determining the variables for encrypting transmitted data [33]. SSL/TLS authentication prevents man-in-the-middle attacks by ensuring the client is communicating with the legitimate API server - and not an attacker impersonating it [39].

The diagram below shows the client establishing a TCP connection using **SYN/SYN-ACK/ACK** before beginning the TLS handshake with the server [34]. When a client attempts to access the Shelfie API using 'https', the client will verify the server's authenticity by requesting an SSL certificate from the server to prove its identity. Once verified, an agreed cryptographic algorithm and session keys are used. The server sends the SSL certificate containing the associated public key. The client and server both end up with the same session key, without it ever actually

being sent over the network. This is what's called a key exchange. Both the server and client use the same session key and safely exchange messages [20], [32].



TLS handshake between client and server

3.3 - Obtaining and Installing an SSL Certificate

The following steps outline the process for obtaining an SSL certificate and installing it:

- Update WHOIS record to match the owner when submitting to the Certificate Authority (CA) [25].
- Generate a Certificate Signing Request (CSR) on the server [25].
- Validate the domain with a CA. The CA is an organisation that signs digital certificates and associated public keys. The certificate includes information such as the organisation, domain and certificate authority [21].
- Upload or bind the issued certificate files on the API server [19], [25]. A wildcard certificate will enable binding this to a connected web app too.

warned to not proceed to unsecured sites with missing or expired certificates - leading to higher bounce rates.

3.4 - Demonstrating Enforcing HTTPS in a Flask application

A 'reverse proxy' such as Nginx or Apache handles SSL encryption [36], [37]. Flask applications can however require HTTPS through rejecting insecure (HTTP) requests and ensuring traffic is routed through a TLS-enabled endpoint, such as through using the Flask-Talisman extension. This can be achieved by setting security headers such as 'Strict-Transport-Security' (HSTS) instructing clients to only communicate over HTTPS. Insecure HTTP requests should be rejected and blocked or redirected to HTTPS. This prevents downgrade attacks where attackers attempt to force clients to use HTTP.

The flask app needs to be deployed behind a server with a valid SSL/TLS certificate. The reverse proxy will perform the TLS handshake when the client sends a request to the Flask API over HTTPS, explained in 3.2 above. It will validate the certificate and forward decrypted requests to the Flask application - terminating the TLS [41].

First, install Flask Talisman: **`pip install flask-talisman`**

The code example shows Flask-Talisman wrapping the Flask app:

```
from flask import Flask
from flask_talisman import Talisman

app = Flask(__name__)
Talisman(app)
```

Flask-Talisman code snippet

Replace the last line with **`Talisman(app, force_https=True)`**

This snippet "**`force_https, default True`**", forces all non-debug connects to https (about HTTPS)" [35].

Testing using tools such as curl, Postman or OWASP ZAP can be achieved by attempting to access the API over HTTP [38]. The use of packet-inspection tools such as Wireshark can verify that traffic is encrypted [22].

4. Authentication and Authorisation

4.1 Theoretical Frameworks

Authentication and authorisation are two complementary security concepts critical to the foundation of an application's access control. Authentication answers the question of who you are, verifying a users' credentials to ensure they prove who they claim to be; for example, fingerprints and passwords [13]. Authorisation determines what the user is given access to do, this can mean resources or actions that are authorised based on their role or permission[14].

In the context of Shelfie, a digital library management system, where users organise personal book collections and write private reviews, upholding these principles are vital. Users must be authenticated to prove they are real account holders, this authorises their access solely to their own books, reviews, and quotes. In the absence of authorisation and authentication Shelfie would be vulnerable to unauthorised access, causing issues where users could view or edit another user's library and reading data. The theoretical framework makes clear that users should have a minimal level of access to complete the main function [14].

4.2 User Authentication

Shelfie displays strong examples of user authentication through password hashing and encompassing validation. During user registration, the system enforces a password policy requiring 8 - 20 characters, including uppercase and lowercase letters, numbers and, special characters. The guidelines of complexity for passwords increases resistance to brute-force attacks [54]. This system validates all user inputs involving email and username formatting, additionally date of birth constraints.

Shelfie stores passwords using Werkzeug's "generate_password_hash", which defaults to scrypt (confirmed via Werkzeug 3.1.8 in requirements.txt), a memory hard hashing algorithm resistant to GPU brute forcing [7]. This is a one way hash, not encryption, the original password can't be recovered, only verified via "check_password_hash". A breach exposes irreversible hashes, not usable passwords [55]. However, Shelfie's authentication application lacks multi-factor authentication (MFA). Utilising MFA, users need a second verification factor such as Microsoft MFA with time based authenticator codes after inputting the correct password to provide extra protection against password compromise [56].

Additionally, Shelfie does not limit login attempts. This gives an attacker infinite attempts to work out the password without risk of throttling. Creating progressively longer delays between failed attempts or IP based limiting would decrease brute-force attacks[57]. All passwords must go through exclusive HTTPS connections, which Shelfie has yet to enforce within its Flask

configuration, exposing a vulnerability where login credentials could be intercepted on decrypted connections through third parties[58].

4.3 Authorisation and Access Control

Authorisation in Shelfie shows as inconsistent, causing significant security vulnerabilities. When a user retrieves their library via the `"/user/<int:user_id>/library"`, the `user_id` is accepted from the URL without authenticated user verification from the account. For instance, if an attacker knows user B's `user_id`, they could manually request viewing of user B's entire book collection, reading progress and personal reviews, the most sensitive data in the Shelfie system[54].

Moreover, Shelfie does not currently use Flask login for session management. After logging in, the app returns a `user_id` maintaining the session, increasing difficulty in verification of which user is making each request. Without its input routes and session cookies authentication cannot be enforced.

4.4 Flask: Login Management

To address the authorisation and session identified in section 4.3, Shelfie should implement Flask Login management. This creates secure sessions, encrypted session cookies are stored in the browser upon login. The configured cookies with secure HTTPS flags and prevents interceptions and XSS. Session timeouts limit exposure - plus authenticated decorators `"@login_required"` ensure only logged in users access these protected endpoints.

5. Protecting your Application Data

5.1 Theoretical Foundations of Database Security

Database security relies on Defence in Depth, assuming perimeter defenses may ultimately fail [42]. Enforcing the Principle of Least Privilege (PoLP) ensures the database does not inherently

trust the application [44]. During the development of Shelfie, the automated integration test suite successfully validated functional pathways; however, it lacked negative testing specifically designed to validate access control boundaries. In this context, using overarching administrative accounts (e.g., root) creates critical vulnerability. Backend authorisation flaws, such as IDOR, can easily bypass functional tests, allowing unauthorised access to compromise master records [51]. To mitigate this, architecture must decouple data structures from access paths via:

Fine-Grained Access Control: Restricting the API exclusively to routine Data Manipulation Language (DML) operations, while explicitly denying structural Data Definition Language (DDL) commands [50] .

Data Abstraction via SQL Views: Granting access strictly to read-only Views, structurally preventing unauthorised engine-level modifications.

5.2 Which best practices would you use to secure your user data?

For Shelfie API, the priorities are least privilege database accounts, encrypted connections, input validation, and proper session handling.

The MySQL account should be scoped to only the permissions Shelfie API needs, not admin level, so a breach doesn't expose the whole database [15]. Connections to MySQL should use TLS 1.2 or higher with certificate verification [15], otherwise credentials and personal data travel as plain text.

Parameterised queries are already used in [db_utils.py](#), but input validation should sit on top as a secondary defence [6], catching malformed data before it reaches the query.

Login also needs a real session token instead of returning [user_id](#) directly, plus rate limiting to stop brute forcing, tying back to the access control gap in section 1.

Passkeys are also worth considering as an alternative to passwords entirely, using public-key cryptography via WebAuthn [49],[46]. Since there's no password to steal or phish, this removes the credential-theft risk covered in Section 4 [45].

Additionally, the project config [47] should use environment variables [48] instead of hardcoding the API key and MySQL password as this reduces the risk of credential exposure, and prevents attackers from obtaining sensitive information through password-hacking techniques:

2. Configure Environment Variables:

Create a config.py file in the root directory and add your Google Books API key and your MySQL credentials

```
GOOGLE_BOOKS_API_KEY = 'your_api_key_here'

db_config = {
    'host': 'localhost',
    'user': 'your_mysql_user',
    'password': 'your_mysql_password',
    'database': 'shelfie'
}
```

The project README showing the config with hard-coded passwords

6. Logging and Monitoring

6.1 Explore the role of logging and monitoring in enhancing the security of the app.

Logging records security-relevant events that occur while an application runs, and NIST identifies log management as a critical part of any security programme [16]. Monitoring builds on this through continuous, automated review of activity and logs to detect suspicious behaviour in real time [17]. Together they give visibility into how an application is used, allowing teams to detect, investigate and respond to threats. The OWASP Top 10 recognises Security Logging and Monitoring Failures as a major risk, as organisations may not realise they have been

compromised until significant damage is done [1]. For Shelfie, which stores users' personal libraries, reviews and quotes, this visibility is vital to protecting that data. However, logging and monitoring involve trade-offs, where they add storage and performance overhead and become a risk themselves if they capture sensitive data. Hence, Shelfie must never write data such as the `/auth/register` password to a log file.

6.2 Discuss the purpose of logging in recording critical events

Logging creates a record of security-related events, allowing suspicious activity to be traced and investigated. Each log entry should include key information such as the time, source, user, action and outcome [26]. For Shelfie, important events to log include user registrations, logins, profile updates and changes to books, reviews and quotes. However, Shelfie currently lacks a structured logging system. Instead, it relies on `print()` statements, such as `print(f"CRITICAL ERROR: {e}")` in `app.py`, which display errors only temporarily and leave no lasting, searchable record to investigate after an attack occurs. In addition, the application exposes internal errors to users through `'details': str(e)`, rather than being securely stored in the server.

6.3 Explain how monitoring tools or services would contribute to this threat detection

Different monitoring tools play different roles in improving an application's security. A Security Information and Event Management (SIEM) platform, such as the open-source ELK Stack, collects and centralises security events from Shelfie's Flask application and MySQL database, detecting suspicious activity and enabling a timely response to potential threats [29]. Fail2Ban strengthens authentication security by scanning login attempts and automatically blocking IP addresses after repeated failed logins, which may indicate a brute-force attack, helping to address the missing rate limiting identified in Section 1. Sentry complements this by capturing application errors in real time, ensuring issues such as those exposed through `str(e)` are securely logged instead of being displayed to users. Managed monitoring services provide an

17

additional layer of protection by using around-the-clock analysis and behavioural analytics to identify unusual activity that simple rule-based systems may miss [30]. For Shelfie, this would include logins from unfamiliar IP addresses or geographic locations that could suggest a compromised account [31].

6.4 Which additional tools would you use?

The most important addition is replacing Shelfie's `print()` statements with Python's built-in logging module, which produces the structured, searchable, timestamped log files that tools such as the ELK Stack, Fail2Ban and Sentry depend on [16]. Since the project is hosted on GitHub, Dependabot can also automatically flag and open pull requests for vulnerable dependencies.

6.5 How would you handle software updates and / or patches?

Keeping software up to date is essential, as outdated dependencies are a common source of security vulnerabilities. Since Shelfie is a Python application, tools such as pip-audit can be used to scan dependencies, including Flask and the MySQL connector, against databases of known weaknesses and flag any that need attention [43]. Once security patches are released, they should be applied promptly and tested before deployment to prevent compatibility issues or other unexpected problems.

REFERENCE LIST (IEEE)

- [1] OWASP, "OWASP Top 10:2021," *OWASP Foundation*, 2021. [Online]. Available: <https://owasp.org/Top10/>. [Accessed: 4 Aug. 2026].
- [2] MITRE, "Common Weakness Enumeration (CWE)," *MITRE Corporation*, 2024. [Online]. Available: <https://cwe.mitre.org/>. [Accessed: 4 Aug. 2026].
- [3] PortSwigger, "Insecure direct object references (IDOR)," *Web Security Academy*. [Online]. Available: <https://portswigger.net/web-security/access-control/idor>. [Accessed: 4 Aug. 2026].

- [4] Pallets, "Debugging Application Errors," *Flask Documentation*, 3.1.x. [Online]. Available: <https://flask.palletsprojects.com/en/stable/debugging/>. [Accessed: 4 Aug. 2026].
- [5] Akamai Technologies, "What Is OWASP API Top 10?," *Akamai Glossary*, 2026. [Online]. Available: <https://www.akamai.com/glossary/what-is-owasp-api-top-10>. [Accessed: Aug. 5, 2026].
- [6] OWASP Foundation, "SQL Injection Prevention Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. [Accessed: 4 Aug. 2026].
- [7] OWASP Foundation, "Password Storage Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed: 4 Aug. 2026].
- [8] OWASP Foundation, "Cross Site Scripting Prevention Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html. [Accessed: 4 Aug. 2026].
- [9] National Institute of Standards and Technology (NIST), "Digital Identity Guidelines," NIST Special Publication 800-63B. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-63b>. [Accessed: Aug. 5, 2026].
- [10] HackTricks, "Homograph attacks," *HackTricks Wiki*, 2026. [Online]. Available: <https://hacktricks.wiki/en/generic-methodologies-and-resources/phishing-methodology/homograph-attacks.html>. [Accessed: Aug. 5, 2026].
- [11] Pydantic, "Pydantic Documentation – Data Validation & Settings Management." [Online]. Available: <https://docs.pydantic.dev/>. [Accessed: 4 Aug. 2026].
- [12] OWASP Foundation, "Input Validation Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html. [Accessed: 4 Aug. 2026].
- [13] M. Kosinski, 'Authentication Vs Authorization', Jun. 2024. Accessed: Aug. 04, 2026. [Online]. Available: <https://www.ibm.com/think/topics/authentication-vs-authorization>
- [14] OWASP, 'Authorization - OWASP Cheat Sheet Series', 2025. Accessed: Aug. 04, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- [15] OWASP, "Database Security Cheat Sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Database_Security_Cheat_Sheet.html. [Accessed: 4 Aug. 2026].

[16] K. Kent and M. Souppaya, *Guide to Computer Security Log Management*, NIST Special Publication 800-92, 2006. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf>. [Accessed: 4 Aug. 2026].

[17] BitLyft, "What Is Security Logging and Monitoring?" [Online]. Available: <https://www.bitlyft.com/resources/what-is-security-logging-and-monitoring>. [Accessed: 4 Aug. 2026].

[18] CrowdStrike, "Types of social engineering attacks: whaling." [Online]. Available: <https://www.crowdstrike.com/en-gb/cybersecurity-101/social-engineering/types-of-social-engineering-attacks/#Whaling> [Accessed: 4 Aug. 2026].

[19] DigiCert, "SSL certificate installation instructions and tutorials." [Online]. Available: <https://knowledge.digicert.com/general-information/ssl-certificate-installation-instructions-and-tutorials> [Accessed: 4 Aug. 2026].

[20] Amazon Web Services, "The difference between HTTPS and HTTP." [Online]. Available: <https://aws.amazon.com/compare/the-difference-between-https-and-http/> [Accessed: 4 Aug. 2026].

[21] Mozilla Developer Network, "Certificate authority." [Online]. Available: https://developer.mozilla.org/en-US/docs/Glossary/Certificate_authority [Accessed: 4 Aug. 2026].

[22] Wireshark, "Introduction to Wireshark." [Online]. Available: https://www.wireshark.org/docs/wsug_html_chunked/ChapterIntroduction.html#ChIntroWhatIs [Accessed: 5 Aug. 2026].

[23] Cloudflare, "What is web application security?" [Online]. Available: <https://www.cloudflare.com/learning/security/what-is-web-application-security/> [Accessed: 4 Aug. 2026].

20

[24] Cloudflare, "What is HTTPS?" [Online]. Available: <https://www.cloudflare.com/learning/ssl/what-is-https/> [Accessed: 4 Aug. 2026].

[25] Kaspersky, "What is an SSL certificate?" [Online]. Available: <https://www.kaspersky.com/resource-center/definitions/what-is-a-ssl-certificate> [Accessed: 4 Aug. 2026].

[26] K. Dunne, "Importance of Logging in Cybersecurity," OSIBeyond, 2024. [Online]. Available: <https://www.osibeyond.com/blog/importance-of-logging-in-cybersecurity/>. [Accessed: 4 Aug. 2026].

[27] OWASP, "Error Handling," *OWASP Cheat Sheet Series*, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html. [Accessed: Aug. 5, 2026].

[28] Open Port Checkers, "What is Port 3306? MySQL Security," *Open Port Checkers Blog*, 2026. [Online]. Available: <https://openportcheckers.com/blog/what-is-port-3306-mysql-security>. [Accessed: Aug. 5, 2026].

[29] M. Vardalachakis, M. Vasilakis, and M. Tampouratzis, "A Comprehensive Analysis of Features, Benefits, Challenges, and Best Practices of Security Information and Event Management (SIEM) Solutions," *Computer Science and Mathematics Forum*, vol. 12, no. 1, art. 18, 2025, doi: 10.3390/cmsf2025012018. [Online]. Available: <https://www.mdpi.com/2813-0324/12/1/18>. [Accessed: 5 Aug. 2026].

[30] M. Martin, "A Practical Guide to Continuous Cyber Security Monitoring," Atlas Systems, 2026. [Online]. Available: <https://www.atlassystems.com/blog/cybersecurity-monitoring>. [Accessed: 5 Aug. 2026].

[31] NetWitness, "Threat Monitoring," 2025. [Online]. Available: <https://www.netwitness.com/cyber-glossary/threat-monitoring/>. [Accessed: 4 Aug. 2026].

[32] DigiCert, "What is SSL/TLS and HTTPS?" [Online]. Available: <https://www.digicert.com/what-is-ssl-tls-and-https> [Accessed: 5 Aug. 2026].

[33] DigiCert, "What is an SSL certificate?" [Online]. Available: <https://www.digicert.com/what-is-an-ssl-certificate> [Accessed: 5 Aug. 2026].

[34] Cloudflare, "What happens in a TLS handshake?" [Online]. Available: <https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/> [Accessed: 5 Aug. 2026].

[35] PyPI, "Flask-Talisman." [Online]. Available: <https://pypi.org/project/flask-talisman/> [Accessed: 5 Aug. 2026].

[36] NGINX, "NGINX web server." [Online]. Available: <https://nginx.org/> [Accessed: 5 Aug.

20

26

].

- [37] Cloudflare, "Reverse proxy." [Online]. Available: <https://www.cloudflare.com/learning/cdn/glossary/reverse-proxy/> [Accessed: 5 Aug. 2026].
- [38] OWASP ZAP, "What does ZAP test for?" [Online]. Available: <https://www.zaproxy.org/faq/what-does-zap-test-for/> [Accessed: 5 Aug. 2026].
- [39] The SSL Store, "Protecting against man-in-the-middle attacks." [Online]. Available: <https://www.thesslstore.com/blog/protecting-against-man-in-the-middle-attacks/> [Accessed: 5 Aug. 2026].
- [40] DataClarity, "The importance of protecting your data: HTTPS and GDPR." [Online]. Available: <https://www.dataclarity.uk.com/2018/02/08/the-importance-of-protecting-your-data-https-and-gdpr/> [Accessed: 5 Aug. 2026].
- [41] Living DevOps, "SSL termination: complete guide 2025." [Online]. Available: <https://livingdevops.com/devops/ssl-termination-complete-guide-2025/> [Accessed: 5 Aug. 2026].
- [42] National Institute of Standards and Technology, "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53, Rev. 5, Sep. 2020. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-53r5>. [Accessed: Aug. 5, 2026].
- [43] Python Packaging Authority, "pip-audit," PyPI. [Online]. Available: <https://pypi.org/project/pip-audit/>. [Accessed: 5 Aug. 2026].
- [44] M. Fathauer, "What is Principle of Least Privilege (PoLP)?," *Ping Identity Blog*, Apr. 14, 2025. [Online]. Available: <https://www.pingidentity.com/en/resources/blog/post/what-is-principle-of-least-privilege-polp.html> [Accessed: Aug. 5, 2026].
- [45] MDN Web Docs, "Web Authentication API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Authentication_API [Accessed: 5 Aug. 2026].
- [46] Curity, "What are passkeys?" [Online]. Available: <https://curity.io/resources/learn/what-are-passkeys/> [Accessed: 5 Aug. 2026].
- [47] Magda UK, "Project Shelfie – Group 5." [Online]. Available: <https://github.com/magda-uk/Project-Shelfie-group-5-project/blob/main/README.md> [Accessed: 5 Aug. 2026].
- [48] Flask Documentation, "Configuring from environment variables." [Online]. Available: <https://flask.palletsprojects.com/en/stable/config/#configuring-from-environment-variables>

[Accessed: 5 Aug. 2026].

22

[49] National Cyber Security Centre (NCSC), "Passkeys." [Online]. Available: <https://www.ncsc.gov.uk/passkeys> [Accessed: 5 Aug. 2026].

[50] Sriram, "Components of SQL," *upGrad Blog*, Jun. 11, 2026. [Online]. Available: <https://www.upgrad.com/blog/components-of-sql/>. [Accessed: Aug. 5, 2026].

[51] N. Manihar, "10 Common Backend Vulnerabilities based on Checkmarx findings (Spring Boot Edition)," *Medium*, Sep. 29, 2025. [Online]. Available: <https://medium.com/@nasiruddinmanihar123/10-common-backend-vulnerabilities-based-on-checkmarx-findings-spring-boot-edition-bd7bec3086d2>. [Accessed: Aug. 5, 2026].

[52] Professor Messer, "Injection Attacks – SY0-701 CompTIA Security+," *Professor Messer*, [Online]. Available: <https://www.professormesser.com/security-plus/sy0-701/sy0-701-video/injection-attacks-sy0-701/> [Accessed: Aug. 5, 2026]

[53] Code First Girls slides, Cybersecurity, session 15, slides 5-6, [Accessed: Aug. 5, 2026]

[54] OWASP, 'Session Management Cheat Sheet', 2025 [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html [Accessed: Aug. 7, 2026]

[55] Code First Girls slides, Cybersecurity, session 5, slides 26;29, [Accessed: Aug. 7, 2026]

[56] M. Kosinski, 'Multi-Factor Authentication', Jan. 2024. [Online]. Available: <https://www.ibm.com/think/topics/multi-factor-authentication> [Accessed: Aug. 7, 2026]

[57] Code First Girls slides, Cybersecurity, session 7, slides 5;10;23, [Accessed: Aug. 7, 2026]

[58] Amazon, 'HTTP Vs HTTPS - Difference Between Transfer Protocols - AWS', 2025. Accessed: Aug. 07, 2026. [Online]. Available: <https://aws.amazon.com/compare/the-difference-between-https-and-http/> [Accessed: 11, Aug. 2026].

[59] Mend.io, "Top open source vulnerabilities," Mend.io Blog. [Online]. Available: <https://www.mend.io/blog/top-security-open-source-vulnerabilities/> [Accessed: 10, Aug. 2026].

[60] SecPortal, "Remote Code Execution (RCE) prevention," *SecPortal Vulnerability Guides*. [Online]. Available: <https://secportal.io/vulnerabilities/remote-code-execution> [Accessed: 10, Aug. 2026].

[61] F. Fernandez, "Top 10 ways to achieve remote code execution (RCE) on web applications," *Medium*, Jul. 1, 2025. [Online]. Available:

<https://fdzdev.medium.com/top-10-ways-to-achieve-remote-code-execution-rce-on-web-applications-d923246b916b> [Accessed: 10, Aug. 2026].

23

[62] Python Packaging Authority (PyPA), "pip-audit: Audits Python environments, requirements files and dependency trees for known security vulnerabilities," *GitHub Repository*, 2026. [Online]. Available: <https://github.com/pypa/pip-audit> [Accessed: 10, Aug. 2026].

[63] GeeksforGeeks, "JSON Web Token (JWT)," *GeeksforGeeks Web Technology*, [Online]. Available: <https://www.geeksforgeeks.org/web-tech/json-web-token-jwt/> [Accessed: 11, Aug. 2026].

[64] Cymulate, "What is a payload in cybersecurity?", *Cymulate Cybersecurity Glossary*, [Online]. Available: <https://cymulate.com/cybersecurity-glossary/payload/> [Accessed: 11, Aug. 2026].

[65] E. LeGault, "The critical role of automated dependency scanning in the modern software development lifecycle," *Leading EDJE Blog*, May 20, 2025. [Online]. Available: <https://blog.leadingedje.com/post/dependencyscanning.html> [Accessed: 11, Aug. 2026].

[66] Kaspersky, "What is a brute-force attack?", *Kaspersky Resource Center*, [Online]. Available: <https://www.kaspersky.com/resource-center/definitions/brute-force-attack> [Accessed: 11, Aug. 2026].

[67] Omada, "What is Role-Based Access Control (RBAC)?", *Omada Blog*, [Online]. Available: <https://omadaidentity.com/resources/blog/what-is-role-based-access-control/> [Accessed: 11, Aug. 2026].

[68] Telerik, "TLS 1.3—What is it and why use it?", *Telerik Blogs*, Apr. 13, 2023. [Online]. Available: <https://www.telerik.com/blogs/tls-1-3-what-is-it-why-use-it> [Accessed: 11, Aug. 2026].

[69] V. Supalov, "Gunicorn and Nginx in a Nutshell," *vsupalov.com*, Apr. 2019. [Online]. Available: <https://vsupalov.com/gunicorn-and-nginx/> [Accessed: 11, Aug. 2026].

[70] Auth0, "Token-Based Authentication Made Easy," Auth0 Learn. [Online]. Available: <https://auth0.com/learn/token-based-authentication-made-easy> [Accessed: 11, Aug. 2026].

[71] SuperTokens, "What is JWT? Understand JSON Web Tokens," SuperTokens Blog, 2024. [Online]. Available: <https://supertokens.com/blog/what-is-jwt> [Accessed: 11, Aug. 2026].

[72] AppSentinels, "How to Secure API Endpoints - Protect & Control Access," AppSentinels Academy, 2025. [Online]. Available: <https://appsentinels.ai/academy/how-to-secure-api-endpoints/> [Accessed: 11, Aug. 2026].

[73] T. Phung, "JWT Tokens VS Session Cookies in Authentication and Authorization," DEV Community, Oct. 22, 2024. [Online]. Available: <https://dev.to/truongpx396/jwt-tokens-vs-session-cookies-in-authentication-and-authorization-1o93> [Accessed: 11, Aug. 2026].

24

[74] B. Frederickson, "Unicode Insanity," benfrederickson.com. [Online]. Available: <https://www.benfrederickson.com/unicode-insanity/>. [Accessed: 12 Aug. 2026].

[75] Flask-Limiter, "Flask-Limiter Documentation," Read the Docs. [Online]. Available: <https://flask-limiter.readthedocs.io/>. [Accessed: 12 Aug. 2026].

[76] Stream, "What is API Rate Limiting?," GetStream.io Glossary. [Online]. Available: <https://getstream.io/glossary/api-rate-limiting/> [Accessed: 12 Aug. 2026].

[77] O. Agboola, E. Akinmade, V. Bemmer, S. Czutor, M. Dominguez, and S. Nasserredine, Project-Shelfie-group-5-project. (2026). GitHub. [Online]. Available:

<https://github.com/magda-uk/Project-Shelfie-group-5-project>. [Accessed: Aug. 12, 2026]. 25